

Data Structures in Python

A visual, practical introduction to the building blocks of Python programming – from lists and dictionaries to stacks, queues, and heaps.

PYTHON FUNDAMENTALS

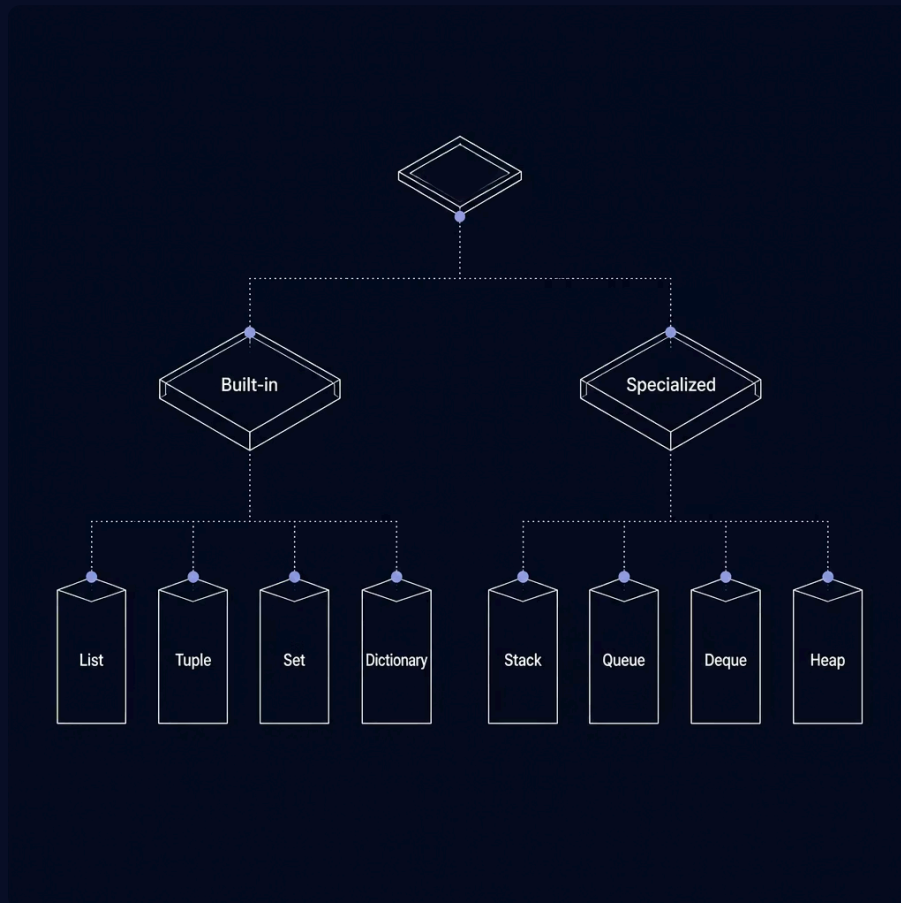
CS UNDERGRADUATE SERIES



Made with GAMMA

Classification of Data Structures

A **data structure** organizes data for efficient access and modification. Python offers **built-in** types for everyday use and **specialized** types for algorithmic tasks.



Real-World Analogy

Think of data structures like storage containers: a **list** is a backpack (ordered, flexible), a **tuple** is a sealed box (fixed), a **set** is a bag of unique marbles, and a **dict** is a labeled filing cabinet.

BUILT-IN

List & Tuple

List

Ordered, mutable, indexed array. Allows duplicates.

```
nums = [1, 2, 3]
nums.append(4)
nums[0] = 10
```

Tuple

Ordered, immutable, indexed. Faster than lists.

```
point = (3, 5)
x, y = point
```

List Methods

Method	Description	Example
append()	Adds element at end	lst.append(10)
extend()	Appends iterable	lst.extend([1,2])
insert()	Inserts at index	lst.insert(1,5)
remove()	Removes value	lst.remove(3)
pop()	Removes & returns last	lst.pop()
sort()	Sorts in place	lst.sort()

Tuple Methods

Method	Description	Example
count()	Counts occurrences	t.count(3)
index()	Finds first index	t.index(5)

Time Complexity

Operation	List	Tuple
Access	O(1)	O(1)
Search	O(n)	O(n)
Append	O(1)*	–

*Amortized.

Applications

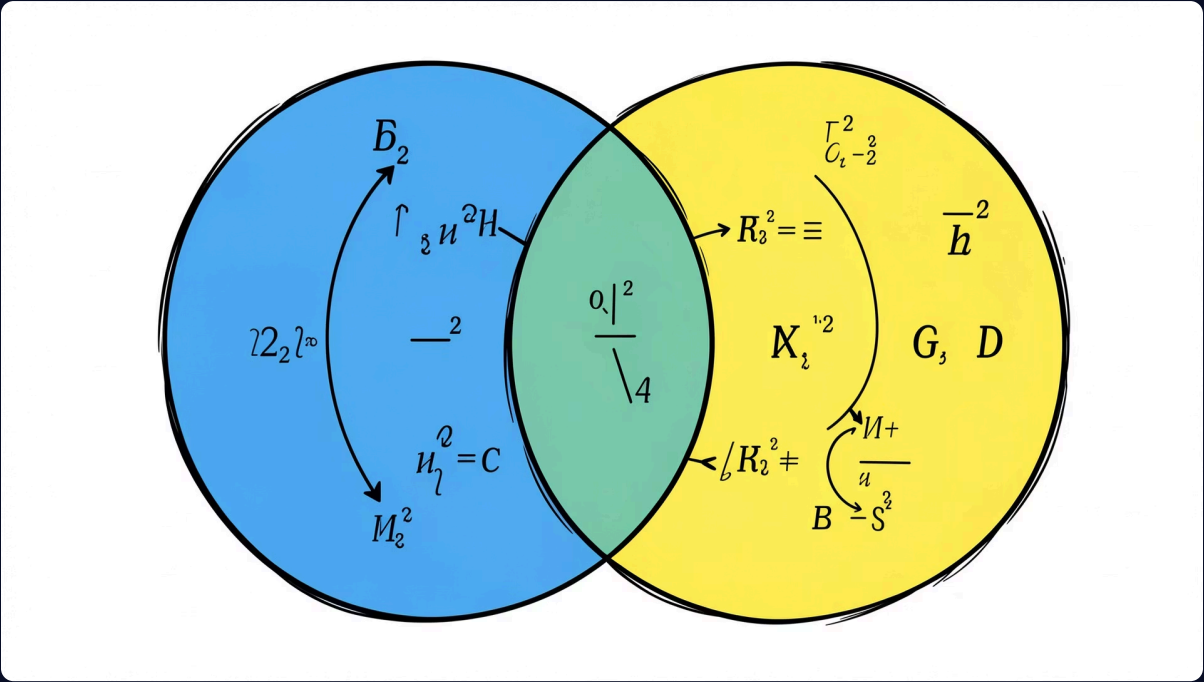
Lists: shopping carts, browser history, dynamic arrays. Tuples: RGB colors, database records, dictionary keys.



Lists are dynamic arrays; tuples are hashable and can be dict keys.

BUILT-IN

Set



Definition

An **unordered** collection of **unique** elements. Mutable but requires hashable items.

Syntax

```
s = {1, 2, 3}
s.add(4)
s |= {5, 6}
```

Key Methods

Method	Description	Example
add()	Adds element	s.add(7)
update()	Merges iterable	s.update([1,2])
remove()	Removes (raises if missing)	s.remove(3)
discard()	Removes (no error)	s.discard(9)
union()	All elements	s1 s2
intersection() &	Common elements	s1 & s2
difference() -	In first, not second	s1 - s2
symmetric_difference() ^	Exclusive elements	s1 ^ s2

Time Complexity

Operation	Complexity
Add / Remove	O(1)
Membership	O(1)
Union / Intersection	O(len(s))

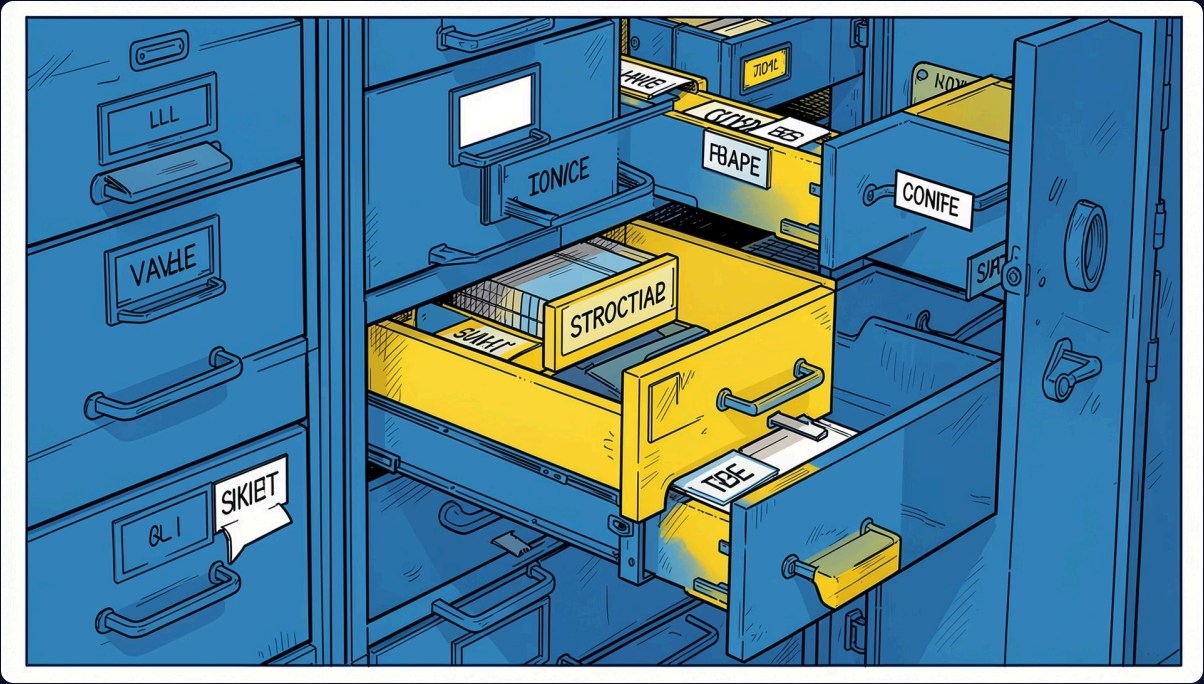
Applications

Deduplication, membership testing, mathematical set operations, filtering unique tags.

Sets use hash tables – membership tests are O(1) vs. O(n) for lists.

BUILT-IN

Dictionary



Definition

An **unordered** (insertion-ordered in 3.7+), **mutable** mapping of **keys to values**. Keys must be immutable.

Syntax

```
person = {"name": "Alice",
"age": 25}
person["age"] = 26
person["city"] = "Boston"
```

Key Methods

Method	Description	Example
get()	Safe access	d.get("name")
keys() / values() / items()	View objects	d.keys()
update()	Merges dict	d.update({"b":2})
pop() / popitem()	Remove key / last pair	d.pop("a")
setdefault()	Set if missing	d.setdefault("x",0)

Time Complexity

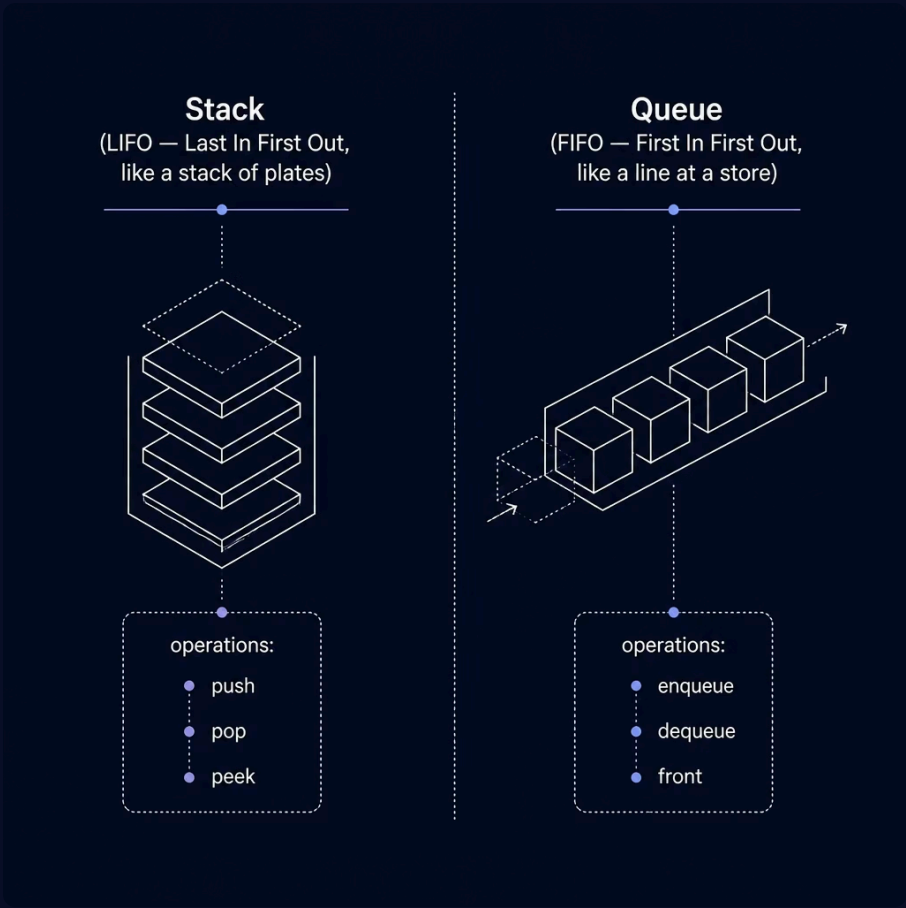
Operation	Complexity
Access / Insert	O(1)
Delete	O(1)
Search (key)	O(1)

Applications

JSON data, configuration files, caching, counting frequencies, database indexing.

Python 3.7+ preserves insertion order – dicts are now ordered by design.

Stack & Queue



Stack

LIFO structure. Use a list or `collections.deque`.

```
stack = []
stack.append(1) # push
stack.append(2)
stack.pop()    # pop → 2
stack[-1]      # peek
```

Stack Methods

Method	Description
<code>append()</code>	Push
<code>pop()</code>	Pop
<code>stack[-1]</code>	Peek
<code>len()</code>	Size

Queue

FIFO structure. Use `queue.Queue` or `collections.deque`.

```
from queue import Queue
q = Queue()
q.put(1)    # enqueue
q.put(2)
q.get()     # dequeue → 1
q.empty()   # check empty
```

Queue Methods

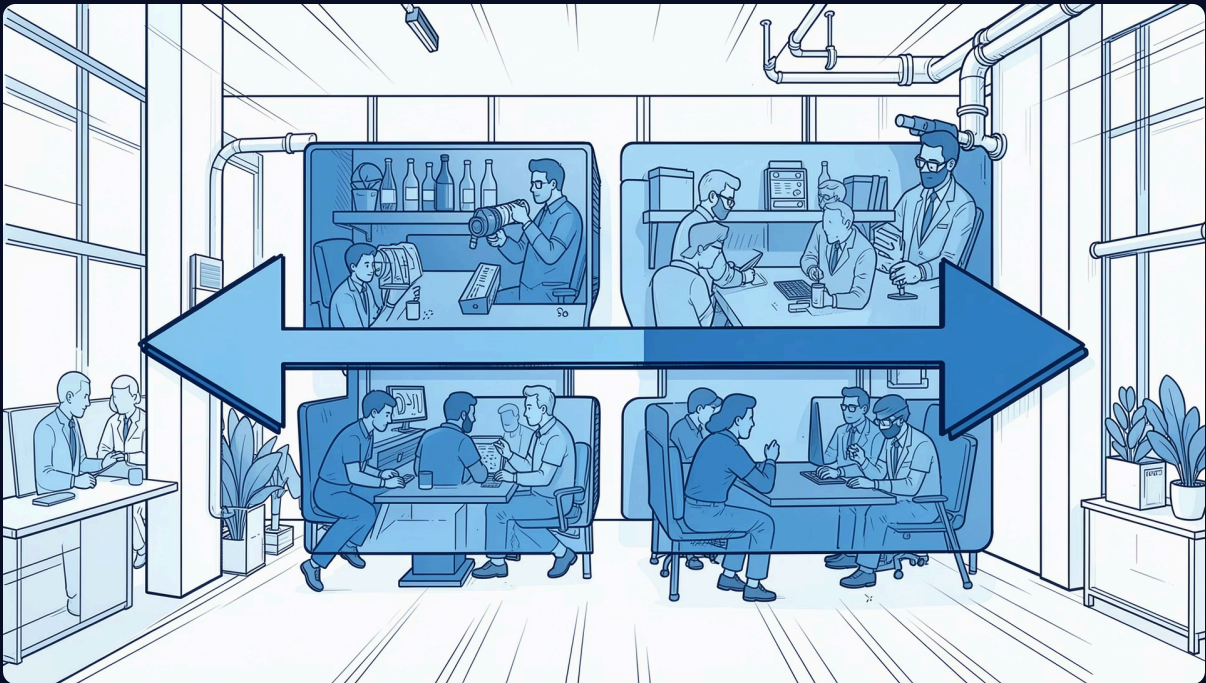
Method	Description
<code>put()</code>	Enqueue
<code>get()</code>	Dequeue
<code>empty()</code> / <code>full()</code>	Status
<code>qsize()</code>	Size

Applications

Stack: undo/redo, recursion, expression parsing. Queue: task scheduling, BFS, print spooling.

💡 Prefer `deque` over list for stacks – $O(1)$ pops from both ends.

Deque



Definition
A **double-ended queue** from collections. Supports $O(1)$ appends and pops from **both ends**.

Syntax

```
from collections import deque
d = deque([1, 2, 3])
d.append(4)      # right
d.appendleft(0)  # left
d.pop()          # → 4
d.popleft()      # → 0
d.rotate(1)      # shift right
```

Key Methods

Method	Description	Example
append() / appendleft()	Add right / left	d.append(5)
pop() / popleft()	Remove right / left	d.popleft()
extend() / extendleft()	Extend right / left	d.extend([1,2])
rotate(n)	Shift n steps	d.rotate(2)

Time Complexity

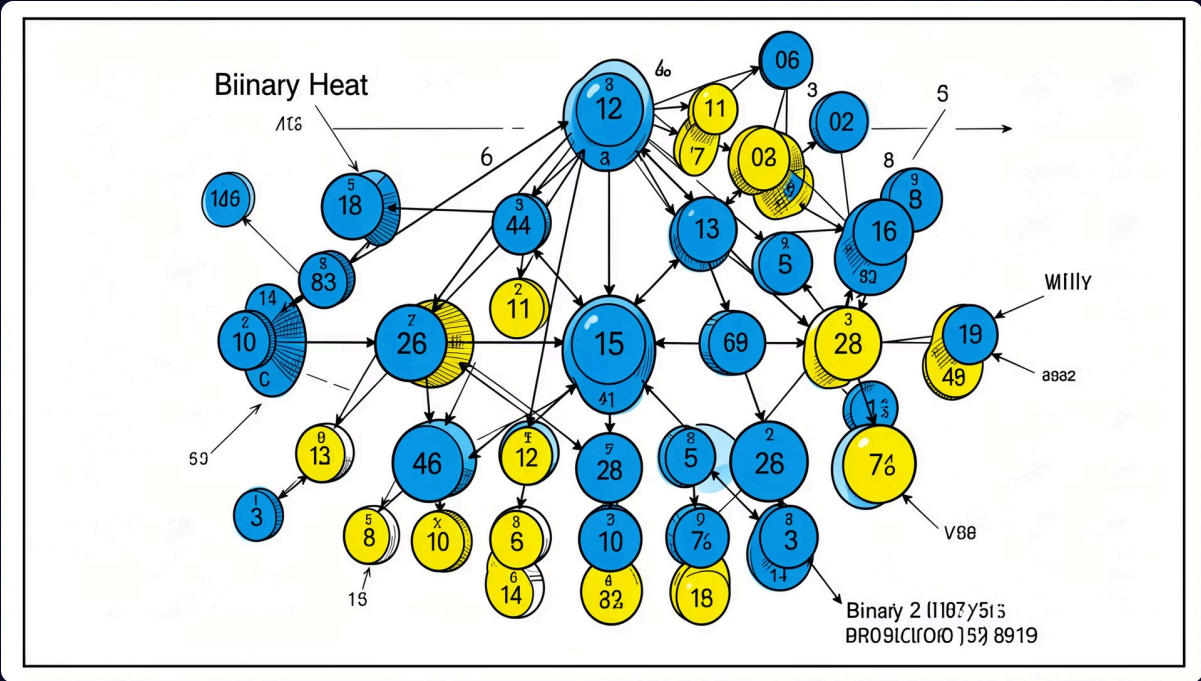
Operation	Complexity
Append / Pop (both ends)	$O(1)$
Random Access	$O(n)$

Applications

Browser history (back/forward), sliding window algorithms, task schedulers, undo/redo stacks.

 Deque is thread-safe for appends/pops – ideal for producer-consumer patterns.

Heap



Definition
A **binary heap** (min-heap by default) via `heapq`. Parent $\leq$ children. Backed by a list.

Syntax

```
import heapq
h = [3, 1, 4]
heapq.heapify(h)    # [1, 3, 4]
heapq.heappush(h, 2) # [1, 2, 4, 3]
heapq.heappop(h)    # → 1
heapq.nlargest(2, h) # → [4, 3]
```

Key Methods

Method	Description	Example
<code>heapify()</code>	Convert to heap	<code>heapq.heapify(h)</code>
<code>heappush()</code> / <code>heappop()</code>	Push / pop min	<code>heapq.heappush(h,5)</code>
<code>heappushpop()</code>	Push then pop min	<code>heapq.heappushpop(h,0)</code>
<code>heapreplace()</code>	Pop then push	<code>heapq.heapreplace(h,6)</code>
<code>nlargest()</code> / <code>nsmallest()</code>	Top/bottom n	<code>heapq.nlargest(3,h)</code>

Time Complexity

Operation	Complexity
Push / Pop	$O(\log n)$
Heapify	$O(n)$
Find Min	$O(1)$

Applications

Priority queues, Dijkstra's algorithm, top-k elements, merge sorted streams, scheduling systems.

For max-heap, negate values: `heapq.heappush(h, -x)`.

Comparison Summary

Structure	Ordered	Mutable	Duplicates	Best Use Case
List	✓	✓	✓	Dynamic sequences
Tuple	✓	✗	✓	Fixed records, dict keys
Set	✗	✓	✗	Uniqueness, membership
Dict	✓ (3.7+)	✓	Keys: ✗	Key-value mapping
Stack	✓	✓	✓	LIFO: undo, recursion
Queue	✓	✓	✓	FIFO: scheduling, BFS
Deque	✓	✓	✓	Double-ended ops
Heap	Partial	✓	✓	Priority queues, top-k



Performance

Right structure = faster code.
Use sets for $O(1)$ lookups,
heaps for priorities.



Built-in + Specialized

Python's `collections` and `heapq`
extend built-ins for advanced
needs.



Match the Problem

Stack for undo, queue for
tasks, dict for configs, heap
for priorities.



Thank You

Questions? Understanding data structures is the first step to writing efficient, elegant Python code.



Keep Learning

Explore `collections`, `heapq`, and `array` modules in the Python docs.



Practice

Try LeetCode or HackerRank problems tagged with arrays, stacks, queues, and heaps.



Python Community

Join [r/learnpython](#), Real Python, or the official Python Discord for support.